

PRACTICAL 9

Q1. File Copy Using Low-Level Linux File I/O

Aim

To implement a file copy utility using low-level Linux system calls such as `open()`, `read()`, `write()`, `lseek()` and `close()`.

Theory

Low-level file operations are performed using system calls. `open()` opens a file, `read()` reads data, `write()` writes data and `close()` closes the file.

Program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int src, dest;
    char buffer[1024];
    int n;

    src = open("source.txt", O_RDONLY);

    if (src < 0) {
        perror("Error opening source file");
        return 1;
    }

    dest = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest < 0) {
        perror("Error creating destination file");
        return 1;
    }

    while ((n = read(src, buffer, sizeof(buffer))) > 0) {
        write(dest, buffer, n);
    }

    close(src);
    close(dest);

    printf("File copied successfully.\n");

    return 0;
}
```

Output

```
amrutha@blue:~$ nano copy.c
amrutha@blue:~$ echo "Hello Linux File I/O" > source.txt
amrutha@blue:~$ gcc copy.c -o copy
amrutha@blue:~$ ./copy
File copied successfully.
amrutha@blue:~$ cat copy.txt
Hello Linux File I/O
```

File copied successfully.

Hello Linux File I/O

Observation

The contents of source.txt were successfully copied to copy.txt.

Result

Thus, a file copy utility was successfully implemented using low-level Linux file I/O system calls.

Q2. I/O Redirection Using dup2()

Aim

To redirect standard input and output using file descriptors and the dup2() system call.

Theory

Linux uses three standard file descriptors:

0 → Standard Input

1 → Standard Output

2 → Standard Error

dup2() can redirect one file descriptor to another.

Program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;

    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("Error opening file");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);

    printf("Hello! This output is redirected to a file.\n");

    close(fd);

    return 0;
}
```

Output

```
amrutha@blue:~$ nano redirect.c
amrutha@blue:~$ gcc redirect.c -o redirect
amrutha@blue:~$ ./redirect
amrutha@blue:~$ cat output.txt
Hello! This output is redirected to a file.
```

Hello! This output is redirected to a file.

Shell Redirection

```
./program > output.txt
```

Input redirection:

```
./program < input.txt
```

Observation

The standard output was redirected from the terminal to output.txt.

Result

Thus, I/O redirection was successfully implemented using dup2().